DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Jingyao Zhang
School of Cyber Science and Engineering
Southeast University
Nanjing, China
zhangjingyao@seu.edu.cn

Abstract— Compiler-instrumented defenses (e.g., stack canaries, Control-Flow Integrity (CFI), and `_FORTIFY_SOURCE`) protect software against security threats, such as memory corruption, control-flow hijacking, and data leakage. However, these defenses can fail silently: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. For example, in CVE-2023-4039, dynamic stack allocation allowed an overflow to bypass the GCC stack canary on AArch64. Existing compiler testing tools fail to detect such bugs. Conventional crash and differential testing only check whether a program produces correct outputs, leaving a fundamental oracle gap when a defense is silently weakened or broken.

In this paper, we present DEFUZZ, an invariant-directed agentic framework for discovering silent failures in compiler-instrumented defenses. DEFUZZ first extracts security invariants from compiler source code, Instruction Set Architecture (ISA) specifications, and historical bug fixes using a hybrid CoT and RAG pipeline. It then maps these invariants to compiler passes and decision branches, locating where defenses are implemented and controlled. To detect violations reliably, DEFUZZ translates these invariants into programmatic bug oracles with formal validation, implementing them as static and dynamic checkers across target ISAs. All bug verdicts are decided strictly by these checkers, with zero LLM involvement in bug adjudication. Guided by the mapped compiler locations, specialized LLM agents perform directed fuzzing to hit targeted defense branches, synthesizing and mutating seeds to trigger invariant violations under oracle evaluation. We evaluated DEFUZZ on GCC and LLVM across multiple ISAs. DEFUZZ discovered [TBD: N] previously unknown silent failures in compiler-instrumented defenses, of which [TBD: M] have been confirmed upstream. Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

I. INTRODUCTION

Compilers deploy a wide range of compiler-instrumented defenses to harden software against security threats, such as memory corruption, control-flow hijacking, and sensitive data leakage. During compilation, these defenses insert runtime bounds checks, enforce stack layout rules, or clear registers to block exploit primitives. However, implementing these defenses accurately inside production compilers like GCC and LLVM is difficult. Defense code must interact with complex optimization passes and translate abstract protection rules into machine instructions across diverse target ISAs. This translation relies on strict assumptions about object bounds, instruction semantics, register conventions, and stack frame layouts. When the implementation of a defense mechanism fails to match these ISA or optimization assumptions, its security protection breaks from within.

When a defense implementation goes wrong, it often causes **silent defense failures**: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. Because the compiled program still runs normally on benign inputs, standard functional tests pass completely, leaving the software exposed to attack. For example, in **CVE-2023-4039** [1], GCC on AArch64 placed the saved frame pointer and return address at the bottom of the fixed stack frame, below the stack canary. When a function used `alloca` or a variable-length array, GCC allocated that dynamic buffer at a lower stack address right below the saved return address. An upward buffer overflow from the dynamic region could then overwrite the saved return address without ever touching the canary.

Existing compiler testing approaches fail to catch such silent bugs due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers (e.g., Csmith [2] and YARPGen [3]) check for compiler crashes or output mismatches across optimization levels. Because a binary with a broken defense still produces correct outputs on benign inputs, crash and differential oracles remain blind when a defense is weakened or broken. The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can reach given target branches, but they cannot tell which branches inside millions of lines of compiler code control a defense. Even after reaching a defense branch, exposing a silent failure requires satisfying defense-specific semantic conditions, such as opaque object bounds or cross-pass length propagation, rather than generic control-flow or data-flow coverage. For instance, in a GCC bug we uncovered (PR 125079 [4]), triggering silent `_FORTIFY_SOURCE` relaxation in the `strlen` pass requires pairing `__strcat_chk(dst, src, objsize)` on an unknown-length buffer with a subsequent `strlen(dst)` call that triggers the rewrite into `__stpcpy_chk`. Random mutators rarely synthesize such coupled defense semantics. Meanwhile, building static analyzers demands heavy manual effort from experts to craft custom rules for each defense mechanism and target ISA, while unconstrained LLM agents

suffer from severe hallucinations and false bug reports.

These barriers raise a central question: how can we systematically detect silent defense failures when neither the buggy compiler nor direct LLM judgment provides a reliable oracle? To answer this question, we present **DEFUZZ**, an invariant-directed agentic framework for testing compiler-instrumented defenses. Our core insight is that although defense implementations vary across backends, their intended security guarantees follow target-independent *security invariants*. DEFUZZ translates these invariants into programmatic bug oracles implemented as ISA-specific static and dynamic checkers, closing the oracle gap. Because LLM-assisted translation can introduce errors, DEFUZZ combines formal validation in oracle construction with offline sampling audits to ensure that the checkers faithfully enforce each invariant. To trigger defense-critical logic, DEFUZZ maps each invariant to its corresponding defense-related compiler branches and guides LLM agents to synthesize inputs that satisfy both branch conditions and invariant preconditions. All bug verdicts remain strictly with deterministic checkers, avoiding both the blind spots of random mutators and LLM adjudication hallucinations.

Building DEFUZZ requires solving three main challenges. The first challenge is establishing reliable security invariants: because the compiler itself has bugs, we cannot trust it. Relying on a compiler's own behavior or implementation code as the ground truth creates circular self-validation. Our key idea is to separate intended security contracts from buggy compiler implementations by anchoring invariants in external ISA manuals, documented design assertions, and confirmed security patches, while abstracting past fixes into transferable failure patterns across defenses. The second challenge is checking these invariants with high accuracy across diverse ISAs, where distinct registers and stack layouts cause direct LLM judgment to hallucinate. DEFUZZ addresses this by translating each invariant into ISA-specific static and dynamic checkers backed by formal validation. Dual static-dynamic cross-checking and differential baseline builds further verify each target, ensuring reliable verdicts with **zero LLM involvement in bug adjudication**. The final challenge is steering test generation to both locate defense branches and satisfy their semantic preconditions without being stripped by early optimizations. DEFUZZ maps each invariant to its controlling compiler passes and branches via call-graph and AST analysis, inserting compile-time probes to trace branch execution. Guided by branch traces and invariant preconditions, LLM agents iteratively synthesize and mutate seeds to expose deep defense flaws.

We evaluated DEFUZZ on GCC and LLVM across x86-64, AArch64, and RISC-V, spanning diverse defenses such as stack canaries, CFI, shadow stacks, register scrubbing, and `_FORTIFY_SOURCE`. DEFUZZ extracted **[TBD: K]** validated security invariants and transferred historical failure patterns across defense mechanisms and backends. During fuzzing, branch-guided seed synthesis hit substantially more target defense branches than unguided baselines, while our formally validated checkers identified violations with high accuracy and avoided the false reports produced by unconstrained LLM agents. Overall, DEFUZZ discovered **[TBD: N] previously unknown silent failures in compiler-instrumented defenses**, of which **[TBD: M] have already been confirmed upstream** (Figure 1), including bugs rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

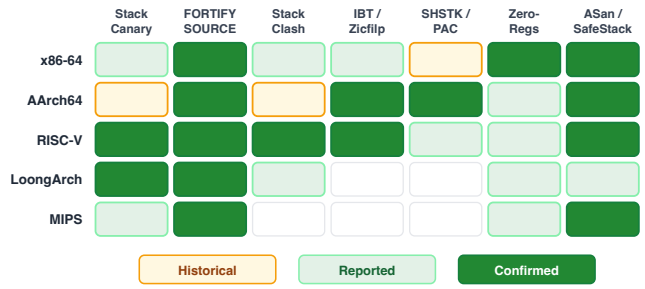


Fig. 1. Partial matrix of silent compiler defense bugs across representative defense mechanisms and target ISAs.

In answering this central question, this paper makes the following contributions:

- **Automated Invariant Extraction:** We decouple security contracts from buggy compiler outputs by extracting security invariants from ISA specifications, design assertions, and historical bug fixes, transferring failure patterns across defenses.
- **Validated Translation into Multi-ISA Programmatic Oracles:** We translate security invariants into formally validated static and dynamic checkers across target ISAs, verifying compiled code with high accuracy and zero LLM involvement in bug adjudication.
- **Call-Graph-Guided Agentic Fuzzing:** We map invariants to defense-related compiler passes and branches, and guide specialized LLM agents with compile-time branch traces to synthesize seeds that satisfy defense triggering conditions.
- **New Silent Defense Bug Discovery:** We evaluate DEFUZZ on GCC and LLVM across multiple ISAs and discover **[TBD: N]** previously unknown silent failures in compiler-instrumented defenses, with **[TBD: M]** confirmed upstream.

II. MOTIVATION

A. Compiler-Instrumented Defenses

Compilers deploy automated defenses to protect software against security threats. Unlike standalone binary rewriters, compiler-instrumented defenses span both middle-end and backend stages of compilation. Middle-end passes analyze object bounds and control flow to insert or optimize high-level checks on the intermediate representation (IR). Backend passes enforce low-level protections by controlling stack frame

layouts, register assignments, and machine instructions across target ISAs.

Preserving security guarantees across this multi-stage pipeline is error-prone. A middle-end optimization can rewrite a defense check under flawed size assumptions, or a backend pass can misplace a stack guard slot. When such bugs occur, the compiler neither crashes nor emits warnings. Instead, it produces a working binary whose security protection is quietly weakened or broken.

B. Motivating Example

Listing 1 shows a silent defense failure in GCC's `_FORTIFY_SOURCE` implementation (PR 125079 [4]) discovered by DeFuzz. This bug had remained hidden for 14 years before upstream maintainers fixed it in the GCC mainline. Under `-D_FORTIFY_SOURCE=2`, GCC replaces unsafe string calls such as `strcat(dst, src)` on the 64-byte buffer `dst` with the bounds-checked builtin `__strcat_chk(dst, src, 64)`. At runtime, this builtin checks that the existing string in `dst` plus the appended `src` fits within 64 bytes, aborting execution on overflow.

```
1  char dst[64];
2
3  size_t append_and_measure(const char *src) {
4    init_opaque(dst); // hides prefix len k (k = 8)
5
6    // 1. Fortified call: __strcat_chk(dst, src, 64)
7    strcat(dst, src);
8
9    // 2. get_string_length() rewrites call to:
10   // __stpcpy_chk(dst + k, src, 64)
11   // BUG: bound stays 64 instead of 64 - k!
12   return strlen(dst);
13 }
```

Listing 1. Silent `_FORTIFY_SOURCE` relaxation in GCC (PR 125079).

In Listing 1, `init_opaque(dst)` initializes the 64-byte buffer `dst` with an 8-byte string ($k = 8$) whose length is unknown at compile time, followed by `strcat(dst, src)` and `strlen(dst)`. Executing `strcat(dst, src)` and `strlen(dst)` naively would scan `dst` twice: once to find the end of the prefix, and again to measure the combined string. To avoid this redundant scan, GCC's `tree-ssa-strlen` pass invokes `get_string_length()` in `tree-ssa-strlen.cc` when it processes the trailing `strlen(dst)` call. Inside `get_string_length()`, the `BUILT_IN_STRCAT_CHK` branch rewrites the earlier `__strcat_chk(dst, src, 64)` into `__stpcpy_chk(dst + k, src, 64)`, which copies `src` to `dst + k` and returns the end pointer to compute `strlen(dst)` directly. However, while GCC advances the destination pointer by k bytes, it fails to subtract k from the size bound, passing 64 instead of $64 - k = 56$. As a result, the runtime check still permits writing 64 bytes starting from `dst + 8`, allowing an 8-byte buffer overflow without triggering an abort.

C. Limitations of Existing Approaches

Existing compiler testing tools fail to detect this vulnerability due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers such as Csmith [2], YARPGen [3], and EMI [5] only check whether the compiler crashes or whether compiled binaries produce consistent outputs on benign inputs. In Listing 1, GCC compiles the code cleanly, and any benign string `src` produces the exact same return value across all optimization levels. Because crash and differential testing do not check whether the binary enforces the reduced bound $(64 - k)$, they remain blind when the defense is weakened.

The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can steer execution toward given target branches, but they cannot tell which branches inside millions of lines of compiler source code control a defense, such as the `BUILT_IN_STRCAT_CHK` branch in `get_string_length()`. Even after reaching that branch, triggering the bug in Listing 1 requires three coupled conditions on `dst`: the buffer must have a known allocation size so GCC emits `__strcat_chk(dst, src, 64)`, its prefix length k must remain unknown at the append site so folding is deferred, and a subsequent `strlen(dst)` call must query the length to trigger the rewrite into `__stpcpy_chk(dst + k, src, 64)`. Random mutators rarely chain an opaque initialization, a fortified append, and a subsequent `strlen` call on the same buffer.

Beyond fuzzing, static analysis requires heavy manual effort from human experts to craft custom rules for each defense mechanism and target ISA. Meanwhile, using LLMs to audit compiler source code end-to-end causes severe hallucinations and false bug reports. To overcome these limitations, a testing framework must locate defense-related compiler branches and guide seed generation toward them. The generated seeds must satisfy the branch conditions so the compiler emits the defense code in the binary. The framework must then run the compiled binary with defense-triggering inputs and use deterministic oracles to check whether the protection is silently weakened or broken.

III. Design of DeFuzz

In this section, we present the design of DeFuzz, an invariant-directed framework for discovering silent failures in compiler-instrumented defenses. Figure 2 illustrates its overall workflow, which consists of four core steps: invariant generation, compiler source mapping, programmatic oracle construction, and agentic directed fuzzing. First, we extract security invariants from compiler source code, ISA specifications, and historical defects using a hybrid CoT and RAG pipeline. With these invariants established, we map them to the relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. Next, we translate these invariants into programmatic bug oracles with formal validation. We implement these oracles as static and dynamic checkers across target ISAs to verify

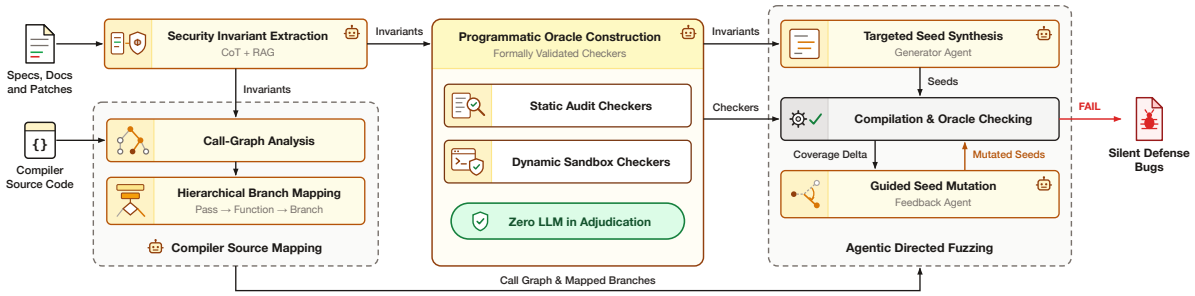


Fig. 2. Overview of DEFUZZ. The framework operates across four core steps: (1) extracting security invariants from compiler source code, ISA specifications, and historical defects via a hybrid CoT and RAG pipeline; (2) mapping invariants to compiler passes and decision branches; (3) translating invariants into formally validated static and dynamic oracles across target ISAs; and (4) performing agentic directed fuzzing, where a generator agent synthesizes probe seeds and a feedback agent steers iterative exploration under programmatic oracle verification.

whether compiled binaries preserve intended defenses, avoiding false reports from model hallucination. Finally, our agentic directed fuzzing coordinates multiple specialized agents: a generator agent synthesizes program seeds designed to violate target invariants, while a feedback agent analyzes execution feedback to guide iterative mutations, with our programmatic oracles evaluating and verifying each candidate. We discuss each step in detail in the following subsections.

A. Security Invariant Generation

To detect silent defense failures, DEFUZZ extracts security invariants that specify the protection rules each compiler defense must enforce. Because the compiler itself may contain bugs, neither its source code nor its compiled outputs can serve as the ground truth. Relying on the current implementation as the reference would cause circular self-validation whenever a bug silently breaks a defense. Instead, DEFUZZ derives security invariants from three external or documented sources: developer design comments and assertions, official ISA and ABI manuals, and confirmed historical bug patches.

Because providing entire compiler directories or architecture manuals at once exceeds the context window and induces hallucinations, DEFUZZ slices inputs along the modular boundaries of optimization passes, target hooks, and machine-description patterns. Each slice pairs a single defense function with its developer comments, assertion macros, and relevant ABI rules. DEFUZZ then applies CoT prompting [6] to analyze one slice at a time, extracting the triggering precondition, the intended protection rule, and the observable violation signal. In the fortified string rewrite from Section II-B, isolating `get_string_length()` alongside its size-tracking comments directs the model to reason over local pointer and bound arithmetic, inferring that advancing a destination pointer by a prefix length must subtract that same length from the remaining size bound.

To uncover analogous flaws across different defenses and backends, DEFUZZ uses a hybrid RAG pipeline [7], [8] to transfer historical bug patterns. Rather than searching with raw patches that only match the original defense, DEFUZZ distills each fix into a mechanism-neutral query containing a root-cause summary, stripped operational tokens, and excluded origin anchors, as shown in Listing 2 for GCC PR 96191 [9]. DEFUZZ retrieves candidate code slices by combining sparse BM25 search with dense vector retrieval via Reciprocal Rank Fusion. BM25 is effective in large software systems because consistent naming conventions and shared macros give underscore-split identifier tokens exact operational meaning, matching functions such as `ix86_zero_call_used_regs`. Dense vector retrieval complements BM25 by matching slices at the root-cause level when distinct defenses implement equivalent logic with different data structures, such as `to_clear_bitmap` in ARM's `cmse_nonsecure_entry_clear_before_return`. After filtering out the excluded origin anchors, the model adapts each homologous slice into a new invariant for the target defense.

```
# Distilled Query from GCC PR 96191
root_cause: "Function exit path returns without
             clearing caller-visible registers"
tokens:     ["epilogue", "caller_saved", "clear",
             "register", "return"]
exclude:    ["stack-protector", "canary"]
# Top Cross-Defense Hits via BM25 + Dense RRF
hit_1_bm25:  "config/i386/i386.cc"
  fn:  "ix86_zero_call_used_regs"
  why: "sub-tokens {zero, call, regs}"
hit_2_dense: "config/arm/arm.cc"
  fn:  "cmse_nonsecure_entry_clear_before_return"
  why: "root cause on register-clear bitmap"
```

Listing 2. RAG query structure and cross-defense retrieval hits.

All invariants extracted via CoT analysis and RAG transfer are stored as structured records in a unified catalog, as shown in Listing 3 for our motivating example. Each record specifies a unique identifier, the target defense, applicable ISAs, the triggering precondition, the protection rule, and the observable violation signal. Keeping this schema independent of compiler source locations allows each invariant to drive both downstream branch mapping and multi-ISA oracle construction.

```
id:       "INV-FORT-014"
defense:  "_FORTIFY_SOURCE"
isas:     ["x86_64", "aarch64", "riscv64"]
precond:  "Fortified append on sized buffer N with
           prefix len k, followed by length query"
rule:     "Rewriting append to offset dst + k must
           reduce remaining size bound to N - k"
signal:   "Bound argument stays > N - k, or runtime
           write past N bytes fails to abort"
```

Listing 3. Structured security invariant record in the catalog.

B. Compiler Source Mapping

With the invariant catalog established, DEFUZZ maps each security invariant to the compiler passes and decision branches that govern its target defense. An invariant specifies what protection a compiled binary must uphold, yet it does not indicate where the compiler implements that logic across millions of lines of C and C++ source code. Lexical search alone is insufficient because a single defense often spans middle-end optimization passes, target hooks, and backend machine descriptions, while providing entire source trees exceeds model context limits. Instead, DEFUZZ equips a mapping agent with a symbol indexer, a call-graph analyzer, and an AST parser. Using these deterministic tools, the agent narrows the codebase along a three-level hierarchy of passes, functions, and conditional branches.

Starting from command-line defense flags or fortified builtin names, the agent queries the symbol indexer and call graph to identify the middle-end or backend pass responsible for the protection. Within that pass, it traverses call-graph edges to locate functions that compute object bounds, lay out stack frames, or insert runtime checks. It then parses each function AST to isolate the conditional expressions and state variables that gate defense emission. Listing 4 shows the resulting mapping for `INV-FORT-014` from Section II-B, tracing fortified string calls to `tree-ssa-strlen`, `get_string_length()`, and the `BUILT_IN_STRCAT_CHK` rewrite branch. Applying the same hierarchy to stack canary invariants maps frame layout rules to `pass_stack_protect`, `expand_used_vars()`, and dynamic allocation predicates such as `has_vla`.

```
# Three-Level Hierarchy Mapping
pass:     "tree-ssa-strlen"
  file:   "gcc/tree-ssa-strlen.cc"
function: "get_string_length"
branch:   "case BUILT_IN_STRCAT_CHK"
  vars:   ["elen", "objsize", "lhs"]

# Injected Compile-Time Branch Probe
DEFUZZ_HIT("fortify#b4e1:hit");
```

Listing 4. Source mapping and branch probe for `INV-FORT-014`.

From each validated mapping, DEFUZZ deterministically instruments a copy of the compiler source tree by inserting a lightweight `DEFUZZ_HIT` probe at each mapped branch, as shown in Listing 4. Each probe carries a unique identifier encoding the defense mechanism and target branch, and DEFUZZ compiles the modified tree into instrumented drivers `GCC_INST` and `CLANG_INST`. Probe logging remains dormant during normal builds and activates only when the `DEFUZZ_LOG` environment variable is set by the fuzzer, writing reached branch identifiers to standard error. Because probes execute strictly inside the compiler process at build time, they never alter the emitted machine instructions, stack layout, or runtime behavior of compiled binaries.

TABLE I
SELECTED HARDWARE AND ABI PARAMETERS IN THE TARGET SPECIFICATION TABLE $\mathcal{T}_i$.

Field	x86-64	AArch64	RISC-V
δ_{grow}	-1 (downward)	-1 (downward)	-1 (downward)
ra_kind	Stack slot	Link reg `x30`	Link reg `ra`
R_{caller}	`rax–r11`	`x0–x18`	`t0–t6, a0–a7`
r_{arg4}	`rcx`	`x3`	`a3`
pad_bytes	`f30f1efa` (`endbr64`)	`5f2403d5` (`bti`)	$\perp$
$\text{align}_{\text{insn}}$	1 byte	4 bytes	2 bytes

C. Programmatic Oracle Construction

To verify whether compiled binaries preserve intended defenses, DEFUZZ translates security invariants from the catalog into programmatic bug oracles implemented as static and dynamic checkers across target ISAs. Online bug verdicts are decided strictly by these compiled checkers, with **zero LLM involvement in bug adjudication**. However, translating invariants into executable checkers offline still relies on LLM assistance, creating a risk of translation errors. Generating raw Go or C analysis code from scratch requires hundreds of lines of parsing boilerplate per rule. Minor coding mistakes, such as matching mnemonics inside assembly comments, inverting stack offset inequalities, or omitting caller-saved scratch registers on unfamiliar backends, directly cause false alarms or missed bugs.

Observation Primitives and Target Specifications. DEFUZZ prevents low-level coding errors by separating declarative check logic from binary parsing and hardware conventions. For a seed compiled on architecture i, deterministic **observation primitives** Ω extract a structured observation state $\omega = \langle \omega_{\text{st}}, \omega_{\text{dyn}} \rangle$ from build artifacts. Static primitives inspect compiler dumps, disassembly, and ELF sections to recover imported symbols, comment-stripped instruction bodies, stack frame offsets, and fortified call arguments. Dynamic primitives execute the binary inside an isolated sandbox to capture post-return register snapshots and boundary overflow signals, while supporting optional Go helper bindings for specialized checks. To handle backend variations without rewriting rules per architecture, DEFUZZ pairs Ω with a read-only **target specification table** $\mathcal{T}_i$, audited once against official ABI manuals. Table I summarizes representative fields of $\mathcal{T}_i$ across x86-64, AArch64, and RISC-V.

Declarative Check IR and Four-State Semantics. Rather than generating raw binary parsers, the offline translator expresses each invariant as a compact **Check IR** specification

over ω and $\mathcal{T}_i$:

$$\phi = \langle \mu,\ I_\phi,\ G_{\text{app}}(\omega, \mathcal{T}_i),\ P_{\text{safe}}(\omega, \mathcal{T}_i) \rangle \tag{1}$$

Here, $\mu \in \{\text{Static}, \text{Dynamic}\}$ selects the inspection modality, I_ϕ lists applicable target ISAs, G_{app} tests whether the compiled binary activates the defense precondition, and P_{safe} asserts the protection property using quantifier-free arithmetic and bounded set predicates. Given observation ω on architecture i, the oracle evaluates ϕ into one of four deterministic verdicts:

$$\text{Eval}(\phi, \omega, \mathcal{T}_i) = \begin{cases} \textbf{ERROR}, & \text{if build or parse fails} \\ \textbf{NA}, & \text{if } i \notin I_\phi \ \vee\ \neg G_{\text{app}} \\ \textbf{PASS}, & \text{if } G_{\text{app}} \ \wedge\ P_{\text{safe}} \\ \textbf{FAIL}, & \text{if } G_{\text{app}} \ \wedge\ \neg P_{\text{safe}} \end{cases} \tag{2}$$

This four-state contract eliminates two major sources of false positives in compiler testing. Build failures or malformed dumps map to **ERROR** rather than spurious bug claims, while seeds that do not activate the target defense precondition evaluate to **NA** rather than misleading passes.

Formal Validation and Differential Anchoring. Before any Check IR rule enters the fuzzing loop, DEFUZZ validates ϕ through three deterministic offline gates without LLM involvement. The first gate performs *type and modality checking* over the AST of ϕ, verifying that every referenced primitive exists in Ω with matching arity, static rules never invoke dynamic execution primitives, and every referenced hardware field is defined in $\mathcal{T}_i$ across all architectures in I_ϕ. The second gate enforces *non-vacuity and boundary consistency* via constraint solving over bounded integer and finite-set domains, confirming that both $G_{\text{app}} \wedge P_{\text{safe}}$ and $G_{\text{app}} \wedge \neg P_{\text{safe}}$ are satisfiable while rejecting tautologies or inverted stack-offset signs relative to δ_{grow}. Once validated, DEFUZZ lowers ϕ into a standalone Go checker and runs *differential baseline anchoring* on minimal probe seeds. Each checker must evaluate to **PASS** on hardened builds such as `-fstack-protector-all` or `-D_FORTIFY_SOURCE=2`, and evaluate to **FAIL** or **NA** on unhardened or fault-seeded builds, complemented by offline human sampling of the concise Check IR specifications.

Static and Dynamic Cross-Checking. During online evaluation, DEFUZZ runs fast static checkers before sandboxed dynamic checkers while sharing cached binary observations across rules. Static checkers resolve structural properties directly from ELF metadata and disassembly. For instance, forward-edge CFI checkers read $\mathcal{T}_i$.pad_bytes and $\mathcal{T}_i.\text{align}_{\text{insn}}$ to scan indirect-callable functions for missing entry landing pads or unintended interior `endbr64` and `bti` gadgets across x86-64 and AArch64, while returning **NA** when pad_bytes $= \perp$.

When defense correctness depends on runtime execution state, dynamic checkers run compiled harnesses inside an isolated sandbox. Consider stack canary epilogue scrubbing in GCC PR 96191 [9], where vulnerable functions compare the stack canary in a scratch register without clearing that register before return:

$$\begin{aligned} &G_{\text{app}}(\omega, \mathcal{T}_i) : \ \texttt{__stack_chk_fail} \in \omega.\text{imports} \\ &P_{\text{safe}}(\omega, \mathcal{T}_i) : \ \forall r \in \mathcal{T}_i.R_{\text{caller}},\ \omega.\text{post_regs}[r] \neq \omega.\text{canary} \end{aligned} \tag{3}$$

Listing 5 shows the lowered post-return register probe. Reading $\mathcal{T}_i.R_{\text{caller}}$ automatically inspects `rax` through `r11` on x86-64, `x0` through `x18` including intra-procedure temporaries on AArch64, and `t0` through `a7` on RISC-V without modifying the rule.

Similarly, for the fortified string rewrite in Section II-B, static inspection checks the bound register $\mathcal{T}_i.r_{\text{arg4}}$ at `__stpcpy_chk` call sites for degenerated `UINT64_MAX` bounds, whereas dynamic probing verifies runtime prefix arithmetic. Listing 6 shows the lowered dynamic harness, which initializes a 64-byte buffer with an 8-byte opaque string and appends a 60-byte payload via `__strcat_chk` followed by `strlen`. Because the 68-byte total write exceeds the 64-byte buffer, an intact defense must raise `SIGABRT` and evaluate to **PASS**, whereas GCC's unadjusted bound of 64 permits the overflow to finish and triggers **FAIL**.

```c
void audit_canary_leak(uintptr_t canary) {
  uintptr_t r0, r1, r2 /* ... */;
  asm volatile(
    "mov %%rax, %0; mov %%rcx, %1; mov %%rdx, %2"
    // ... expanded from T_i.R_caller per ISA
    : "=r"(r0), "=r"(r1), "=r"(r2)
  );
  if (r0 == canary || r1 == canary /* ... */)
    emit_verdict(FAIL, "Canary leaked");
  else
    emit_verdict(PASS, "Canary cleared");
}
```

Listing 5. Lowered post-return register probe for canary scrubbing.

```c
void test_fortify_strcat(const char *src60) {
  char buf[64];
  init_opaque_8bytes(buf); // opaque prefix k = 8
  setup_sigabrt_handler(); // trap fortify abort

  // Rewritten to __stpcpy_chk(buf + 8, src60, 64)
  __builtin___strcat_chk(buf, src60, sizeof(buf));
  sink = __builtin_strlen(buf); // triggers fold
  emit_verdict(FAIL, "Overflow unchecked");
}

void sigabrt_handler(int sig) {
  emit_verdict(PASS, "Overflow trapped");
  _exit(0);
}
```

Listing 6. Dynamic boundary probe for `_FORTIFY_SOURCE` bounds.

D. Agentic Directed Fuzzing

With the invariant catalog, mapped compiler branches, and validated checkers in place, DEFUZZ launches its online directed fuzzing campaign, summarized in Algorithm 1. Allowing unconstrained LLM agents to explore compiler codebases freely leads to nondeterministic tool loops, lost provenance, and unverified bug claims. Instead, DEFUZZ enforces a strict

Algorithm 1 Agentic Directed Fuzzing per Invariant

Require: Invariant ϕ, mapped path $\mathcal{M}_\phi$, spec $\mathcal{T}_i$, budget B
Ensure: Minimized bug PoCs $\mathcal{F}$ on versioned blackboard $\mathcal{B}$

$\mathcal{F} \leftarrow \emptyset$; $g \leftarrow \mathsf{InitGuidance}(\phi, \mathcal{M}_\phi)$
for $t = 1$ **to** B **do**
  $s_t \leftarrow \mathsf{GeneratorAgent}(\phi, \mathcal{M}_\phi, g)$
  $(b_t, \tau_t) \leftarrow \mathsf{CompileInst}(s_t, i)$ {`DEFUZZ_LOG=1`}
  $v_t \leftarrow \mathrm{Eval}(\phi, \Omega(b_t, s_t), \mathcal{T}_i)$
  $\mathcal{B}.\mathsf{Append}(s_t, \tau_t, v_t)$
  if $v_t = \mathbf{FAIL}$ **then**
    $s^* \leftarrow \mathsf{MinimizerAgent}(s_t, \phi, \mathcal{T}_i)$
    $\mathcal{F} \leftarrow \mathcal{F} \cup \{s^*\} \cup \mathsf{Amplify}(s^*, \phi, I_\phi)$
    break
  else if $\mathsf{HitTarget}(\tau_t, \mathcal{M}_\phi) \wedge \mathsf{Stalled}(\mathcal{B})$ **then**
    $\phi \leftarrow \mathsf{RefineAndValidate}(\phi, \mathcal{B})$; $\mathsf{Replay}(\phi, \mathcal{B})$
  else
    $g \leftarrow \mathsf{FeedbackAgent}(\phi, \mathcal{M}_\phi, \tau_t, v_t)$
  end if
end for
return $\mathcal{F}$

topological workflow through a deterministic orchestrator. At startup, the orchestrator loads validated invariants into a queue and allocates a fixed iteration budget B per target invariant and architecture. Each round executes a fixed sequence of targeted seed synthesis, compilation with `GCC_INST` or `CLANG_INST`, two-phase oracle evaluation, and trace-guided mutation. All intermediate artifacts, including synthesized seeds, compiler flags, `DEFUZZ_HIT` branch traces, and oracle verdicts, are recorded on an append-only versioned blackboard $\mathcal{B}$, enabling deterministic replay of every test seed and bug verdict.

Branch-Guided and Optimization-Resistant Seed Synthesis. In each round, the Generator Agent synthesizes C or C++ seeds designed to reach the mapped path $\mathcal{M}_\phi$ from Section III-B and satisfy the triggering precondition of ϕ. Conventional random program generators fail at two stages. Early middle-end passes such as Dead Code Elimination, constant folding, and dead-store removal routinely strip unused buffers and redundant calls before defense passes execute. Even when code survives early cleanup, exposing a silent defense flaw requires coupled semantic constraints across multiple statements. In the fortified string bug from Section II-B, triggering the faulty rewrite inside `get_string_length()` requires combining a statically sized destination buffer with a runtime-opaque prefix length, a fortified `strcat` append, and a trailing `strlen` query on the same buffer. Guided by $\mathcal{M}_\phi$ and ϕ, the Generator Agent constructs compact multi-statement programs satisfying these coupled conditions. Rather than applying blanket `volatile` qualifiers that would suppress the target optimization pass itself, the agent places opacity barriers selectively at function boundaries. It derives dynamic allocation sizes from runtime `argc` inputs, marks external helper and sink functions with `noinline` and `noclone` attributes, and inserts inline `asm` memory clobbers to keep defense-relevant stack frames and bounds live through backend lowering.

Trace-Driven Divergence Feedback and Oracle Refinement. Each synthesized seed is compiled by the instrumented driver with `DEFUZZ_LOG` enabled, producing a candidate binary b_t alongside a compile-time trace τ_t of reached defense branches. When $\mathrm{Eval}(\phi, \omega, \mathcal{T}_i)$ returns a non-failing verdict, DEFUZZ passes compiler diagnostics, checker evidence, and τ_t to the Feedback Agent, which diagnoses three distinct failure modes. If evaluation returns $\mathbf{ERROR}$, the agent repairs invalid target flags, missing declarations, or ABI type mismatches. If evaluation returns $\mathbf{NA}$ or $\mathbf{PASS}$ while target branches are absent from τ_t, the agent compares τ_t against the mapped hierarchy $\mathcal{M}_\phi$ to pinpoint where compilation diverged. For instance, τ_t may reveal that compilation entered `expand_used_vars()` with `has_vla` unset, or returned from `get_string_length()` prior to `BUILT_IN_STRCAT_CHK`. From this divergence, the Feedback Agent issues structural mutation directives specifying which variable qualifiers, buffer declarations, or call sequences the Generator Agent should adjust next. When repeated rounds hit the deepest target branch in $\mathcal{M}_\phi$ while still returning $\mathbf{NA}$, the orchestrator triggers bounded Check IR refinement on G_{app}, re-validates the updated rule through the offline gates of Section III-C, and replays recorded seeds from $\mathcal{B}$.

Oracle-Locked Minimization and Lateral Amplification. When a checker returns $\mathbf{FAIL}$, the oracle confirms a silent defense failure from deterministic machine evidence alone. DEFUZZ then invokes a Minimizer Agent that couples C-Reduce [10] with the programmatic oracle to isolate a minimal root-cause trigger. Standard reduction predicates based on compiler crashes or exit codes often collapse silent defense bugs into unrelated undefined behavior. Instead, DEFUZZ recompiles every reduced candidate with the instrumented driver and accepts a reduction step only if the target checker still evaluates to $\mathbf{FAIL}$ while preserving the target branch hit in τ. Once minimization produces a clean Proof-of-Concept seed s^*, the orchestrator executes a bounded lateral amplification step before advancing to the next invariant. Conditioning on s^*, the Generator Agent systematically varies surrounding data types, struct nesting, alignment attributes, optimization levels, sibling builtins such as `__strcpy_chk` and `__mempcpy_chk`, and peer target architectures in I_ϕ to uncover related silent defense flaws sharing the same root cause.

IV. IMPLEMENTATION

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation, routing, build, and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

REFERENCES

[1] MITRE, "CVE-2023-4039: GCC AArch64 stack protector bypass with variable-length arrays," https://nvd.nist.gov/vuln/detail/CVE-2023-4039, 2023.

[2] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[3] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.

[4] GCC Bugzilla, "Bug 125079: tree-ssa-strlen get_string_length: __strcat_chk to __stpcpy_chk drops bound subtraction," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125079, 2026.

[5] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[8] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[9] GCC Bugzilla, "Bug 96191: aarch64 stack_protect_test canary leak," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191, 2020.

[10] J. Regehr *et al.*, "C-Reduce: A C and C++ program reducer," https://github.com/csmith-project/creduce, 2012.